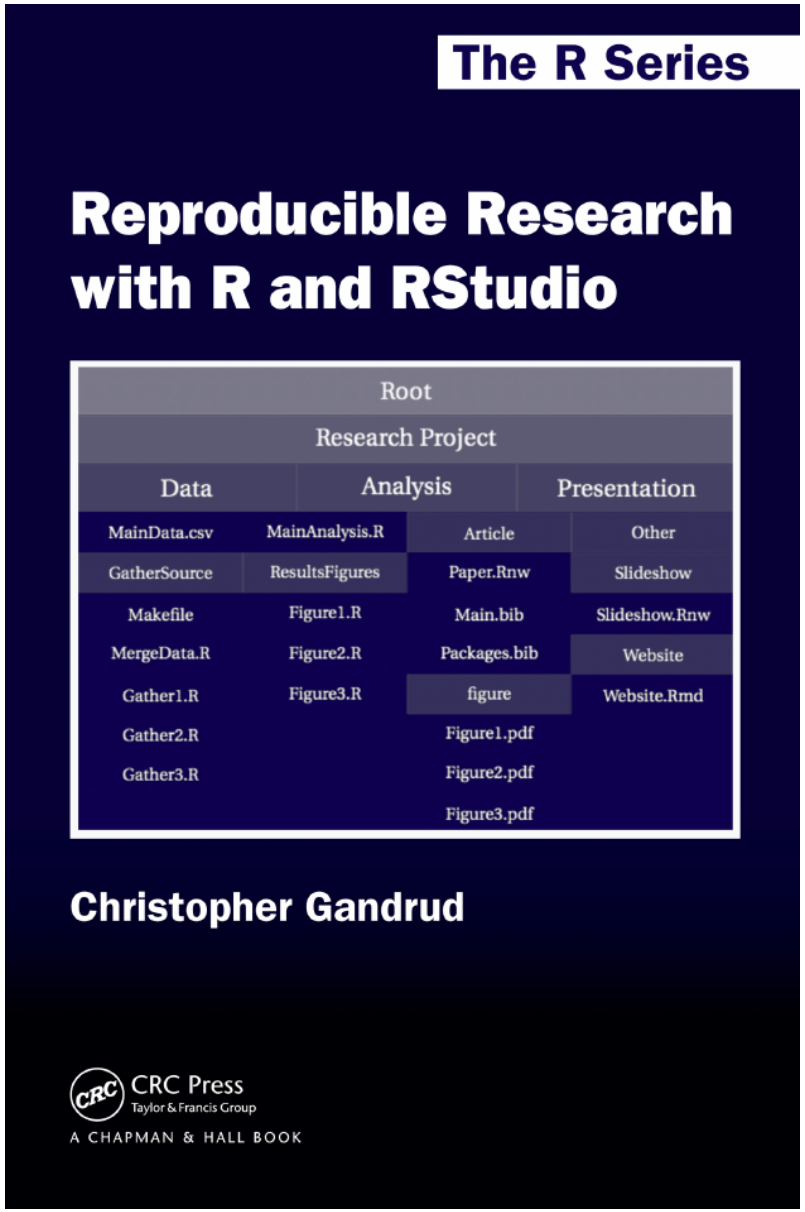
Scientific Programming
Chapter 2 - Numerical Errors and Reproducible Research

Jean-Sébastien Lerat

2023-2024

Main Reference



Introduction

Reasons:

- when need to write up results
- when a journal/team requires data and code
- when another researcher/colleague/manager asks if they can use the data to reproduce the findings.

numerous versions stored across multiple folders of the data, analyses

$\Rightarrow$ difficult and time consuming

Content

1. Introduction
2. Numerial Errors
   - Underflow
   - Overflow
   - Rounding Error
3. Solve errors
   - Bouding values
   - Changing Scale
   - Example
4. Reproductibility
   - Good Practices
   - Norms (functions API)
   - Randomness
   - Multithreading/process
5. Exercise

Numerical Errors

Underflow

Underflow error

This error occurs when a number close to zero is considered as zero.

Numerical Errors

Underflow

Simple example:

```
               _
   _       _ _(_)_     |  Documentation: https://docs.julialang.org
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | | |/ _` |  |
  | | |_| | | | (_| |  |  Version 1.10.0 (2023-12-25)
 _/ |\__'_|_|_|\__'_|  |
|__/                   |

julia> x = 1e-64
1.0e-64

julia> x = x*x
9.999999999999999e-129

julia> x = x*x
9.999999999999998e-257

julia> x = x*x
0.0

julia>
```

Numerical Errors

Underflow

More complex example: $g(x) = \dfrac{e^x}{e^C}$

```
               _
   _       _ _(_)_     |  Documentation: https://docs.julialang.org
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | | |/ _` |  |
  | | |_| | | | (_| |  |  Version 1.10.0 (2023-12-25)
 _/ |\__'_|_|_|\__'_|  |
|__/                   |

julia> f(x,c) = exp(x) / exp(c)
f (generic function with 1 method)

julia> g(x) = f(x,-745)
g (generic function with 1 method)

julia> g(1)
Inf

julia> exp(-745)
5.0e-324

julia> exp(-746)
0.0

julia>
```

Numerical Errors

Overflow

Overflow error

This error occurs when a very large number is approximated as ∞. Can result in a not-a-number (`NaN`) value.

Numerical Errors

Overflow

Simple example:

```
               _
   _       _ _(_)_     |  Documentation: https://docs.julialang.org
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | | |/ _` |  |
  | | |_| | | | (_| |  |  Version 1.10.0 (2023-12-25)
 _/ |\__'_|_|_|\__'_|  |
|__/                   |

julia> x = 1e64
1.0e64

julia> x = x*x
1.0e128

julia> x = x*x
1.0000000000000002e256

julia> x = x*x
Inf

julia>
```

Numerical Errors

Rounding Error

Caused by:

digital computer limitations
: using an incorrect data type for a variable.

the programmer applied numerical rounding
: using a function like `round(•)` and `ceil(•)`.

Numerical Errors

Rounding Error

Simple example:

```
               _
   _       _ _(_)_     |  Documentation: https://docs.julialang.org
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | | |/ _` |  |
  | | |_| | | | (_| |  |  Version 1.10.0 (2023-12-25)
 _/ |\__'_|_|_|\__'_|  |
|__/                   |

julia> (1e32 + 1 ) - 1e32
0.0

julia>
```

Numerical Errors

Rounding Error

Complex example: $f(x,y,z) = \dfrac{20x + 5y^5 + 2z}{z-2}^3 - 1000\,\log(x)$

```
               _
   _       _ _(_)_     |  Documentation: https://docs.julialang.org
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | | |/ _` |  |
  | | |_| | | | (_| |  |  Version 1.10.0 (2023-12-25)
 _/ |\__'_|_|_|\__'_|  |
|__/                   |

julia> f(x,y,z) = ((20*x + 5*y^5 + 2*z)/(z-2))^3 - 1000 * log10(x)
f (generic function with 1 method)

julia> function f_round(x,y,z)
               x = round(x, base=2, digits=1)
               y = round(y, base=2, digits=1)
               z = round(z, base=2, digits=1)
               f(x,y,z)
       end
f_round (generic function with 1 method)

julia> f(4.56,3.57,5.12)
8.897601860453526e8

julia> f_round(4.56,3.57,5.12)
7.503402074542557e8
```

Content

1. Introduction
2. Numerial Errors
   - Underflow
   - Overflow
   - Rounding Error
3. Solve errors
   - Bouding values
   - Changing Scale
   - Example
4. Reproductibility
   - Good Practices
   - Norms (functions API)
   - Randomness
   - Multithreading/process
5. Exercise

Solve errors

Bouding values

Bounds values to avoid underflow

```
julia> x = 1/3 - 1 + 2/3 + 1
0.9999999999999999

julia> if( x == 1) println("One") else println("Not one") end
Not one

julia> if( abs(x-1) < 1e-15) println("One") else println("Not one") end
One
```

Solve errors

Changing Scale

Change scale to avoid overflow

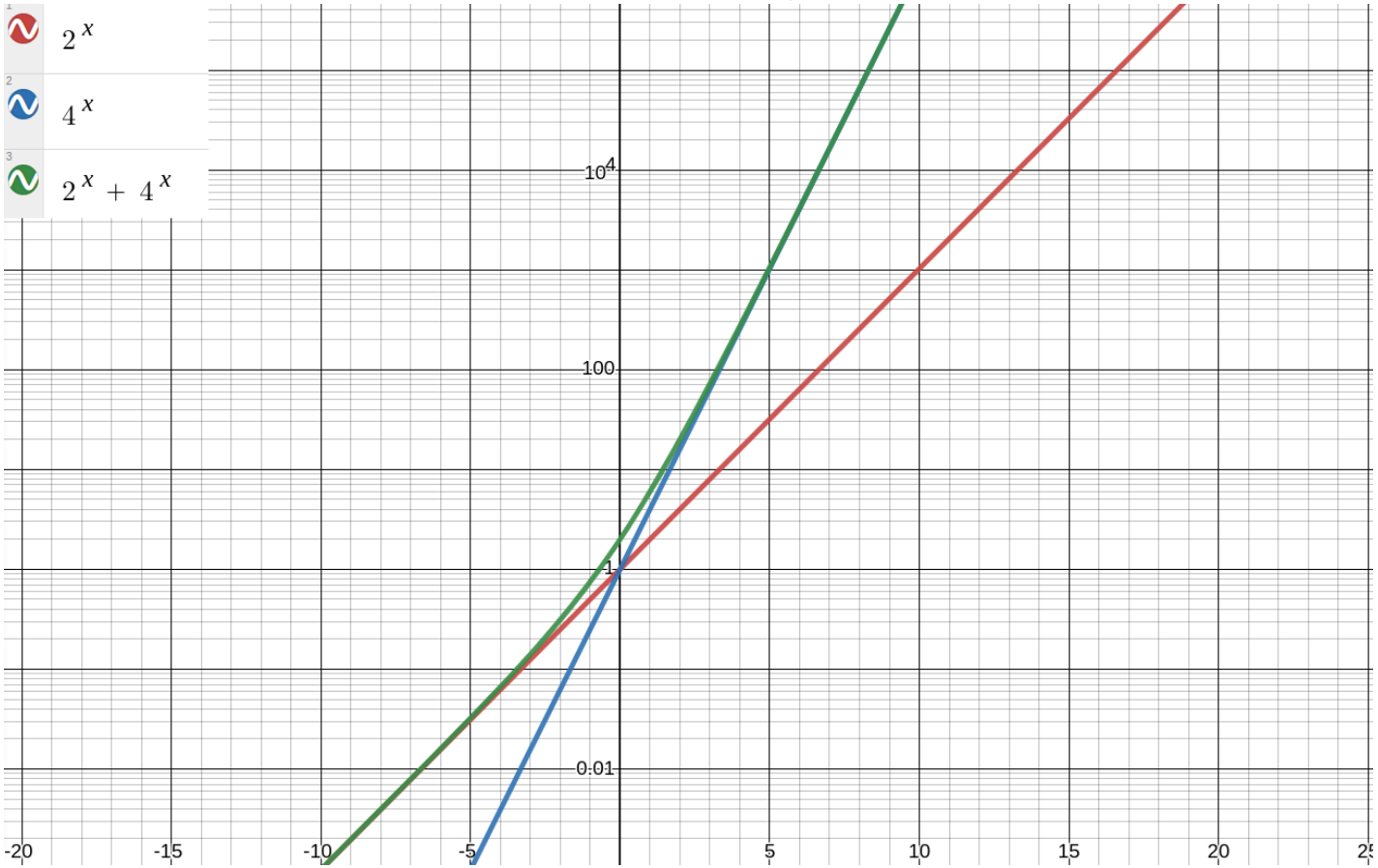
Example: from linear

Solve errors

Changing Scale

Change scale to avoid overflow

Example: to log scale



Solve errors

Example: `sqrt(•)`

How to implements `sqrt(•)` ?

Any idea?

Solve errors

Example: `sqrt(•)`

Mathematics ...

Given a (the squared number) and the x^2:

$$\text{Which } x \text{ solves} \qquad x^2 = a$$

Example:

$$x^2 = 16$$

The positive solution is $x = 4$.

Rewrite $x^2 = a$ by $x^2 - a = 0$ and find the root (i.e. when $f(x) = 0$).

Solve errors

Example: `sqrt(•)`

There are severals methods, focus on

Newton-Raphson method

is a root-finding algorithm that produces successively better approximations of the roots of a real-valued function.

Conditions: $f(\bullet)$ must be continuous and differentiable on the search interval.

$f(x) = x^2 - a$ is continous and differentiable $f\prime(x) = 2x$

Solve errors

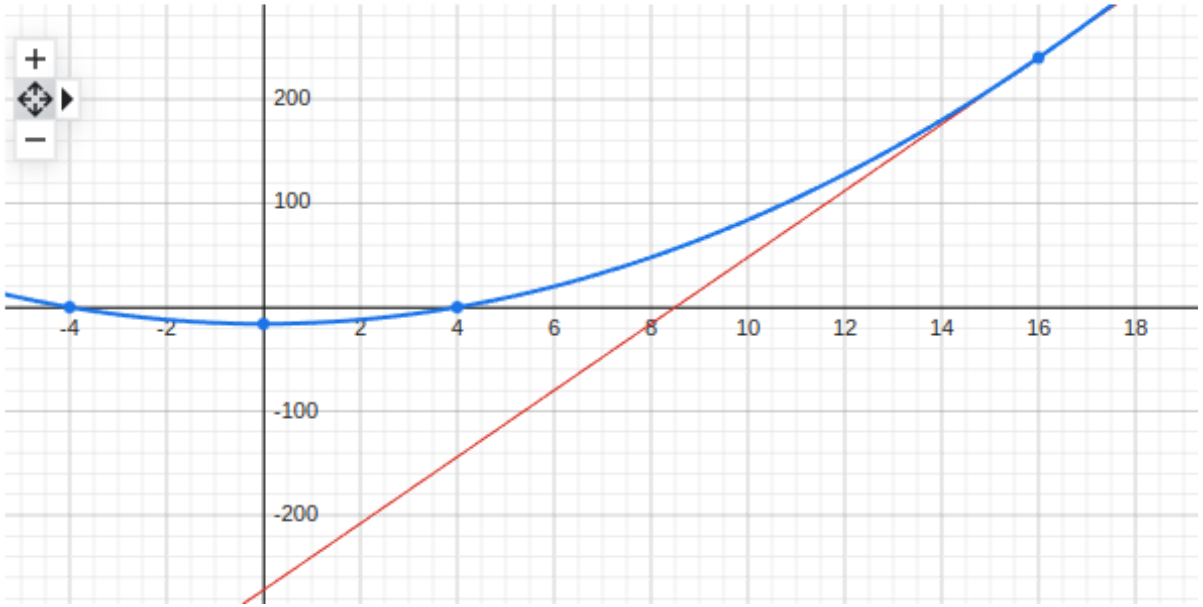
Example: `sqrt(•)`

The Newton-Rephson method uses the tangent at a

which is defined by $\quad g(x) = f(x) + f\prime(a)(x - a)$

to iterate from one approximation to another, starting at a:

Graph for $x^2 - 16$, $16^2 - 16 + 2 \cdot 16(x - 16)$



$$f(x) = x^2 - 16, \qquad a = 16$$

Solve errors

Example: sqrt(•)

Let's implement sqrt(•):

```
julia> function my_sqrt(a,precision=1e-15,limit_iter=100)
               f(x)        = x*x - a
               fp(x) = 2*x
               let iter = 0, x = a
                       while (abs(x) > precision) && (iter < limit_iter)
                               x                 = -f(x)/fp(x) + x
                               iter        += 1
                       end
                       return x
           end
       end
my_sqrt (generic function with 3 methods)

julia> print(my_sqrt(5))
2.23606797749979
julia> print(sqrt(5))
2.23606797749979
```

Solve errors

Example: `sqrt(•)`

Log-scale can go further:

```
julia> my_sqrt(9e153)
7.099748146989106e123

julia> my_sqrt(9e154)
NaN

julia> exp(0.5*log(9e154))
3.000000000000003e77
```

Log-scale: mutliplication becomes addition, power becomes multiplication.

$$\sqrt{(}x) = x^{\frac{1}{2}}$$

Content

1. Introduction
2. Numerial Errors
   - Underflow
   - Overflow
   - Rounding Error
3. Solve errors
   - Bouding values
   - Changing Scale
   - Example
4. Reproductibility
   - Good Practices
   - Norms (functions API)
   - Randomness
   - Multithreading/process
5. Exercise

Reproductibility

Good Practices

Organize your research process:

1. Document everything.
2. Everything is a (text) file.
3. All files should be human readable.
4. Explicitly tie your files together.
5. Have a plan to organize, store, and make your files available.

Document everything

Use a notebook (see Apache Zeppelin[1]) to comment, store and execute code.

[1] https://zeppelin.apache.org

Reproductibility

Good Practices

Organize your research process:

1. Document everything.
2. Everything is a (text) file.
3. All files should be human readable.
4. Explicitly tie your files together.
5. Have a plan to organize, store, and make your files available.

Everything is a (text) file

include data, analysis code, the write up of results, and explanations of these files. Prefer simple format (like *.csv*, *.tex*) instead of file format that change over time (like *.xslx*, *.docx*).

Reproductibility

Good Practices

Organize your research process:

1. Document everything.
2. Everything is a (text) file.
3. All files should be human readable.
4. Explicitly tie your files together.
5. Have a plan to organize, store, and make your files available.

All files should be human readable

Treat all of your research files as if someone who has not worked on the project will, in the future, try to understand them.

Reproductibility

Good Practices

Organize your research process:

1. Document everything.
2. Everything is a (text) file.
3. All files should be human readable.
4. Explicitly tie your files together.
5. Have a plan to organize, store, and make your files available.

Explicitly tie your files together

it is important to make the links between your files explicit.

Reproductibility

Good Practices

Organize your research process:

1. Document everything.
2. Everything is a (text) file.
3. All files should be human readable.
4. Explicitly tie your files together.
5. Have a plan to organize, store, and make your files available.

Have a plan to organize, store, and make your files available

From the beginning of your research process you should have a plan for organizing your files and a way to make them accessible, for example: GitHub[1].

[1] https://www.github.com

Reproductibility

Norms (functions API)

Reproductibility

Randomness

Always use and initialize a
pseudorandom number generator (PRNG) with a **seed**

Counter to security (e.g. video games, cryptography) **do not use entropy** or cryptographic PNG.

A such algorithm: MersenneTwister

Take **care of devices** (CPU PRNG versus GPU PRNG), they can differ!

Reproductibility

Randomness

```
julia> using Random

julia> seed = 1337
1337

julia> prng = Random.MersenneTwister(seed)
MersenneTwister(1337)

julia> rand(prng,Int,1,3)
1×3 Matrix{Int64}:
 -5017376553968290583  -3384053142747626775  -1885838122248145070

julia> rand(prng,Int,1,3)
1×3 Matrix{Int64}:
 -8725636656299489794  3794610726871079455  -2226209838258710519

julia> prng = Random.MersenneTwister(seed)
MersenneTwister(1337)

julia> rand(prng,Int,1,3)
1×3 Matrix{Int64}:
 -5017376553968290583  -3384053142747626775  -1885838122248145070
```

Reproductibility

Randomness

From `https://cuda.juliagpu.org/stable/development/kernel/`

CPU PRNG versus CUDA PRNG:

1. the default RNG uses global state (undefined behavior with multiple instances)
2. (CUDA) kernels automatically seed the RNG with a unique seed passed from the host ⇒ multiple invocations produce different results
3. manual seeding with `Random.seed!`. CUDA wrap can handle multiple threads that share the PRNG state ⇒ one thread must set the seed.
4. to continue the random sequence, need to use `Random.seed!` to set the seed **and** the counter value.

Reproductibility

Multithreading/process

Shared random state

```
julia -threads 4 myfile.jl
```

```julia
using Random

function run()
    sleep(0.42)
    println(rand())
end

Random.seed!(1337)

tasks = [Task(run) for i in 1:Threads.nthreads()]
for task in tasks
    schedule(task)
end
```

Reproductibility

Multithreading/process

Distinct PRNG state

```julia
using Random

function run()
    global prng
    sleep(0.42)
    println(rand(prng[Threads.threadid()]))
end

Random.seed!(1337)
prng = [rand(1:Int(0xffffffff)) for i in 1:Threads.nthreads()]
prng = [Random.MersenneTwister(rnd) for rnd in prng]

tasks = [Task(run) for i in 1:Threads.nthreads()]
for task in tasks
    schedule(task)
end
```

Content

1. Introduction
2. Numerial Errors
   - Underflow
   - Overflow
   - Rounding Error
3. Solve errors
   - Bouding values
   - Changing Scale
   - Example
4. Reproductibility
   - Good Practices
   - Norms (functions API)
   - Randomness
   - Multithreading/process
5. Exercise

Exercise

Taylor series of order n to approximate a function $f(\bullet)$ in the neighbourhood of a, f is at least n times derivable:

$$f(x) \approx f(a) + \frac{(x-a)^1}{1!} f'(a) + \frac{(x-a)^2}{2!} f''(a) + \ldots + \frac{(x-a)^n}{n!} f^n(a) + \varepsilon$$

When $x \to a$, the error is $\varepsilon = 0$

1. Propose an implementatoin of the `exp(•)` function based on the Taylor series of order $n = 18$ in the neighbourhood of $a = 0$
2. What is the error of e^{10}?
3. Create a (clean) reproductible script that run 20 threads, each generate a random real $\in [0; 10]$ and output the result of your `exp(•)` function.